Lab 10 — SQL Injection (SQLi)

> **SY0-701 Objective 2.3** — Explain various types of vulnerabilities
>
> **Domain 2** — Threats, Vulnerabilities, and Mitigations (22%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: bypass authentication on a deliberately vulnerable login page, then patch it so the same input fails.

You'll build: a tiny vulnerable PHP/SQLite login app and its prepared-statement fix, side by side.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1.

> **Legal note:** The target runs only on `127.0.0.1` inside your own playground. Running SQL-injection tools against systems you do not own is an offence under Singapore's Computer Misuse Act. Keep everything local.

Background

- **SQL injection** happens when untrusted input is concatenated into a query so the database parses it as *syntax*, not *data* (OWASP A03: Injection).
- A **tautology** payload such as `' OR '1'='1` forces a WHERE clause always true; a trailing `--` comments out the rest of the query.
- The authoritative fix is **parameterised queries** (prepared statements) that bind input strictly as data.
- Supporting controls: least-privilege database accounts, input validation and a WAF (which buys time but never replaces the code fix).

Steps

1. Install the PHP/SQLite stack

```
apt-get update -qq && apt-get install -y sqlite3 php-cli php-sqlite3 curl
```

2. Create the database and a vulnerable login page

The query concatenates user input directly — the vulnerability.

```
sqlite3 app.db "CREATE TABLE users(u TEXT, p TEXT); INSERT INTO users
VALUES('admin','s3cret');"
cat > login.php <<'EOF'
<?php
$db = new SQLite3('app.db');
$u = $_GET['u']; $p = $_GET['p'];
$sql = "SELECT * FROM users WHERE u='$u' AND p='$p'";   /* VULNERABLE */
$r = $db->query($sql);
```

```
echo $r && $r->fetchArray() ? "LOGIN OK\n" : "DENIED\n";
EOF
```

3. Start the local PHP server

```
php -S 127.0.0.1:8080 -t . >/tmp/php.log 2>&1 &
sleep 1 && echo "server up"
```

4. Confirm a normal login works

```
curl -s 'http://127.0.0.1:8080/login.php?u=admin&p=s3cret'
curl -s 'http://127.0.0.1:8080/login.php?u=admin&p=wrong'
```

```
LOGIN OK
DENIED
```

5. Bypass authentication with an injection

The payload closes the username quote and comments out the password check.

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--+&p=anything"
```

```
LOGIN OK      # authentication bypassed without the password
```

6. Patch with a parameterised query

Bind the input with placeholders so it can never be parsed as SQL.

```
cat > login.php <<'EOF'
<?php
$db = new SQLite3('app.db');
$st = $db->prepare("SELECT * FROM users WHERE u=:u AND p=:p");   /* FIXED */
$st->bindValue(':u', $_GET['u'], SQLITE3_TEXT);
$st->bindValue(':p', $_GET['p'], SQLITE3_TEXT);
$r = $st->execute();
echo $r && $r->fetchArray() ? "LOGIN OK\n" : "DENIED\n";
EOF
```

7. Verify the injection now fails

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--+&p=anything"
curl -s 'http://127.0.0.1:8080/login.php?u=admin&p=s3cret'
```

```
DENIED        # injection rejected
LOGIN OK      # legitimate login still works
```

Test it

Confirm the before/after behaviour difference.

```
B=$(curl -s "http://127.0.0.1:8080/login.php?u=admin'--+&p=x")
echo "After patch, injection returns: $B"
echo "$B" | grep -q DENIED && echo "PATCH OK — bound parameters defeat the injection"
```

Expected: before the patch the injection returned `LOGIN OK`; after the patch it returns `DENIED` while the valid password still logs in.

Troubleshooting

Symptom	Fix
`Address already in use`	A server is already running; `pkill -f 'php -S'` then retry step 3.
Injection returns `DENIED` even on the vulnerable page	URL-encode the payload (`'` as `%27`) or keep the `--+` trailing space-comment.
`SQLite3 class not found`	Install `php-sqlite3` and restart the PHP server.

Challenge

Extend the vulnerable page to be dumpable by a UNION-based injection (reveal the table of usernames), then confirm the prepared-statement version blocks it. Add a least-privilege note: what database rights should the web account actually have?

Reflection

Objective 2.3 names SQLi as a web-based vulnerability. Why is input validation alone (a block list of quotes) weaker than parameterised queries as the primary control?

Exam tips

- Injection is a *class* (SQL, OS command, LDAP, XPath) with one root cause: untrusted input reaching an interpreter.
- The primary, authoritative fix is parameterised/prepared statements — not escaping or a WAF.
- `' OR '1'='1` is a tautology; a trailing `--` comments out the rest of the query.
- Least-privilege DB accounts limit the blast radius if a flaw slips through.

References

- OWASP SQL Injection — https://owasp.org/www-community/attacks/SQL_Injection
- OWASP Query Parameterization Cheat Sheet — https://cheatsheetseries.owasp.org
- CWE-89 SQL Injection — https://cwe.mitre.org/data/definitions/89.html
- PortSwigger Web Security Academy (SQLi) — https://portswigger.net/web-security/sql-injection

---